STANSE: Bug-finding Framework for C Programs

Jan Obdržálek, Jiří Slabý and Marek Trtík

Faculty of Informatics, Masaryk University, Brno, Czech Republic
{obdrzalek,slaby,trtik}@fi.muni.cz

Abstract. Stanse is a free (available under the GPLv2 license) modular framework for finding bugs in C programs using static analysis. Its two main design goals are applicability on large software projects like Linux kernel and extendability with new bug-finding techniques with a minimal effort. Currently there are three bug-finding algorithms implemented within Stanse: AutomatonChecker checks properties described in an automata-based formalism, ThreadChecker detects deadlocks among multiple threads, and ReachabilityChecker looks for unreachable code. Stanse has been tested on the Linux kernel, where it has found dozens of previously undiscovered bugs.

1 Introduction

During the last decade, bug-finding techniques based on static analysis have finally come of age. One of the papers to really stir interest was [2], showing that static analysis can efficiently find many interesting bugs in a real-world code. This work eventually led to a successful commercial tool called Coverity [7]. Over the years, several other successful tools, like CodeSonar [6] or Klocwork [9], appeared. However, such fully-featured tools are neither free to obtain, nor is their code available (e.g. for developing new algorithms or tailoring the existing tools to specific tasks). The existing free tools are usually severely limited in what they can do (e.g. Uno [12], Sparse [11], Smatch [10]). One notable exception is FindBugs [4, 8], a successful tool working on Java code. Stanse is intended to fill this gap for C language. It can be seen in two ways:

I think, reference to Coccinelle is missing here

1. Stanse is a robust framework (written predominantly in Java) for implementing diverse static analysis algorithms. An implemented algorithm can be immediately evaluated on large real-world software projects written in C as the framework is capable to process such projects (for example, it can process the whole Linux kernel). An implementation of such an algorithm within the framework is called *checker*.
2. As Stanse contains three checkers already, it can be also seen as a working static analysis tool.

The rest of the paper is structured as follows. Section 2 describes the functionality provided by the framework, while Section 3 is devoted to the three

existing checkers. Section 4 presents some results of Stanse on the Linux kernel. The last section summarizes the basic strengths of Stanse and mentions some directions of its future development.

2 Framework Functionality

The functionality provided by the Stanse framework can be roughly separated into several stages. First, the framework reads in the configuration including the list of source files to be analysed and checkers to be used. The source files are then parsed into a set of *internal structures*, namely *abstract syntax trees (ASTs)*, *control-flow graphs (CFGs)*, and *call-graphs*. Next, checkers are run on these structures and error reports are collected and presented.

2.1 Selecting and Parsing Source Files

There are several ways to tell Stanse what source files to analyse. Besides the standard possibilities (a single given file, all files from a given directory, all files listed in a given file), Stanse can also derive all the necessary information from a project Makefile. In this case, Stanse also remembers compiler flags for preprocessing purposes. This functionality is inspired by the Sparse [11] tool.

Stanse can process source files written in C, more specifically the ISO/ANSI C99 standard and most of the GNU C extensions. Before parsing, the sources are preprocessed by the standard GNU C preprocessor. The code is then parsed into the mentioned internal structures: a call-graph, a CFG for each function, and an AST for each code piece corresponding to a node of some CFG. All these structures can be dumped in a textual or graphical form.

As mentioned earlier, the design of the framework is geared towards analysing large software projects. Such projects consist of hundreds or thousands of modules (compilation units). In practice, it is often impossible to store all the corresponding internal structures in the memory. The Stanse framework therefore applies automatic *streaming* of the internal structures, currently on a module basis. Instead of parsing all source modules at the beginning, a module is streamed in only when Stanse needs to access some internal structure corresponding to the module. In other words, the internal structures are constructed in a lazy manner. If the memory occupied by internal structures exceeds a given limit, some internal structures have to be freed before another module is streamed in. The structures to be freed are selected by an LRU (least-recently-used) approach: Stanse discards all internal structures of the module whose structures are not accessed for the longest time. If the discarded structure is later accessed again, the corresponding module is streamed back in. Both laziness of internal structures and streaming are completely invisible to checkers.

2.2 Traversing the Parsed Code

Although a checker can process the internal structures in an arbitrary way, most of checkers walk through CFGs using some standard strategy. To prevent

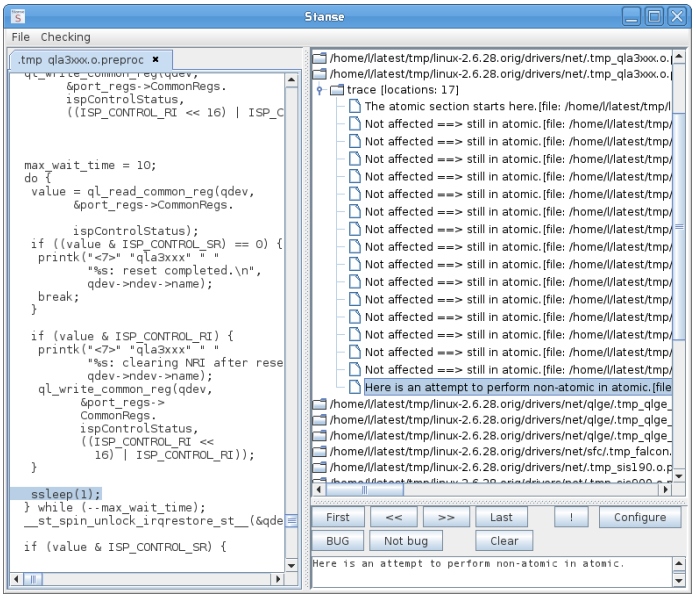

Fig. 1. The error trace GUI browser.

unnecessary reimplementations, the most important and heavily used traversal methods are implemented directly inside the framework. With this functionality, one can implement a new checker (or its part) by specifying

- whether it should go through CFGs forwards or backwards, breadth-first or depth-first,
- if the interprocedural walk-through should be performed or not (if not, the function calls are ignored), and
- a method (callback) to be called for each visited node in a CFG.

This makes implementation of new algorithms extremely simple.

2.3 Processing Errors

Once a checker finds an error, it reports back a commented *error trace* (a path in the analysed code) demonstrating this error. Stanse provides several possibilities to present the error traces: they can be printed to the console, displayed using a built-in error trace GUI browser (see Figure 1), or saved to an external file in XML format. This XML file has a wide variety of possible applications. For example, we supply a tool transforming the XML file into an SQLite database complemented with a web interface allowing to browse errors in the database via a web browser. Using the web browser or the built-in error GUI browser, one can mark errors as real bugs or false-positives. Stanse also provides various statistics of errors like number of errors per file and checker, frequency of errors of the same kind, percentage of false-positives (based on user feedback), etc.

2.4 Other Features

The framework provides some other features. For example, it supports parallel execution of more checkers (in threads). Note that the internal structures are shared by all running checkers. Further, Stanse can run in a command line mode or via a fully featured GUI. If properly configured, Stanse can be used as a "push-the-button" tool. Due to the design of the framework, Stanse can be adapted to handle other programming languages by writing appropriate parsers. Finally, the Stanse framework is basically a Java library, so it can be easily transformed into a static analysis plug-in for some IDE (e.g. Eclipse or NetBeans).

3 Checkers

In this section we describe the three currently available checkers.

AutomatonChecker is heavily influenced by [2]. It takes, as an input, a set of finite-state automata that describe some properties which we want to check. Properties like locking discipline, interrupt management, null pointer dereference, dangling pointers and many others can be handled this way.

Compared to the implementation described in [2] and [3], our technique differs in several aspects. In particular, we do not use metacompilation, automata are not input-language specific (a pattern matching is used instead), and the interprocedural analysis is done in the context of a single input file.

ThreadChecker aims to check for possible deadlocks in concurrent programs. The technique is based on the notions of locksets of [5] and deadlock detection by looking for cycles in *resource allocation graphs (RAGs)*. ThreadChecker first tries to identify the parts of the code which can run in parallel, as different threads. Then it builds a set of *dependency graphs* for each such a thread. A dependency graph statically represents the possible locksets during one execution of a thread. Dependency graphs are then combined and transformed to RAGs. If some RAG contains a cycle, an error is reported.

ReachabilityChecker searches CFGs for unreachable nodes. These are then reported as warnings or errors, depending on importance (e.g. superfluous semicolons are less important than unused branch). The primary goal of ReachabilityChecker is to demonstrate the simplicity of a new checker implementation using the framework features described in Subsection 2.2: the code of the checker has less than 200 lines including the mentioned error/warning classification and many strings and comments.

4 Results on Linux Kernel

We have several reasons to choose the Linux kernel to test Stanse: the kernel presents a large and freely available codebase, it fully exercises most features of ISO/ANSI C99 and GNU C extensions, it is under constant development (new bugs are appearing), and absence of bugs are of great concern.

We applied Stanse with the three checkers described in the previous section to kernel version 2.6.28. AutomatonChecker uses three automata describing the following kinds of errors:

- incorrect pairing of functions (imbalanced locking, reference counting errors)
- bugs in pointer manipulation (null dereference, dangling pointers, etc.)
- deadlocks caused by sleeping inside spinlocks or interrupt handlers

The running time of Stanse on a common desktop machine with 2 cores and 4 GiB of memory is under 100 minutes. The memory usage of the Java process oscillates between 400 and 1000 MiB. The number of errors reported by the checkers can be found in Table 1. Let us note that ReachabilityChecker actually detected 751 errors, but 720 of them are of a low importance (including 696 superfluous semicolons) and they are omitted from our statistics.

We have manually analysed all the reported errors and mark them as real bugs or false positives (with an exception of 60 errors reported by AutomatonChecker where we are not able to decide in a short time whether it is a false positive or not). The numbers of real bugs, false positives and the ratio of real bugs to all classified reported errors can be also found in Table 1. The overall ratio of real bugs to all classified errors is not high: 44.7%. However, Stanse in the current version does not have any thorough false-positive filtering technique.

More than 70 of the 169 found bugs have been reported to kernel developers and fixed in the following kernel releases (the other bugs have been independently discovered and reported by someone else or the incorrect code disappeared from the kernel before we finished our evaluation of reported errors). Some of the reported bugs remained undiscovered for seven years (for example, see `http://lkml.org/lkml/2009/3/11/380`).

We have reported another 50 bugs found by Stanse in the subsequent versions of the kernel. This number is increasing every month.

Checker	**Automaton**	**Reported err.**	**Bugs**	**False pos.**	**Ratio**
AutomatonChecker	Pairing	266	65	143	31.3 %
	Pointers	86	48	37	56.5 %
	Deadlocks	35	16	18	47.1 %
ThreadChecker		20	9	11	45.0 %
ReachabilityChecker		31	31	0	100.0 %
Overall		438	169	209	44.7 %

Table 1. Stanse results on Linux kernel version 2.6.28.

5 Conclusions and Future Work

Stanse is a free Java-based framework for simple implementation of bug-finding algorithms based on static analysis. The framework can process large-scale software projects written in ISO/ANSI C99. Stanse does not currently use any new techniques – its novelty comes from the fact that (to our best knowledge) there is no other open-source framework with comparable applicability and efficiency. Stanse is available at `http://stanse.fi.muni.cz`. We note that more than 130 bugs found by Stanse have been reported to and confirmed by Linux kernel developers already.

Future Work We plan to improve the framework in several direction. We are currently working on C++ support. Further, we plan to provide Stanse in the form of a plug-in for Eclipse and NetBeans IDEs. A lot of work can be done in the area of automatic false-alarm filtering and error importance classification. Independently of new features development, we would like to speed up the framework as well. More precisely, we intend to replace the current parser written in Java by an optimized parser written in C, to replace the XML format of internal structures by a more succinct representation, to add a support for function summaries, etc.

Acknowledgements Jan Kučera is the author of the ThreadChecker. We would like to thank Linux kernel developers, and Cyrill Gorcunov for Stanse alpha testing and useful suggestions. All authors are supported by the research centre Institute for Theoretical Computer Science (ITI), project No. 1M0545.

References

1. T. Ball, B. Hackett, S. Lahiri, S. Qadeer, and J. Vanegue. Towards scalable modular checking of user-defined properties. *Verified Software: Theories, Tools, Experiments*, pages 1–24, 2010.
2. D. Engler, B. Chelf, A. Chou, and S. Hallem. Checking system rules using system-specific, programmer-written compiler extensions. In *OSDI'00*, pages 1–16, 2000.
3. S. Hallem, B. Chelf, Y. Xie, and D. Engler. A system and language for building system-specific, static analyses. In *PLDI'02*, pages 69–82. ACM, 2002.
4. D. Hovemeyer and W. Pugh. Finding bugs is easy. In *OOPSLA'04*, pages 132–136. ACM, 2004.
5. J.W. Voung, R. Jhala, and S. Lerner. RELAY: static race detection on millions of lines of code. In *ESEC-FSE'07*, pages 205–214. ACM, 2007.
6. CodeSonar. `http://www.grammatech.com/products/codesonar/`.
7. Coverity. `http://www.coverity.com/products/`.
8. FindBugs. `http://findbugs.sourceforge.net/`.
9. Klocwork. `http://www.klocwork.com/products/`.
10. Smatch. `http://smatch.sourceforge.net/`.
11. Sparse. `http://www.kernel.org/pub/software/devel/sparse/`.
12. Uno. `http://spinroot.com/uno/`.